

What is DMA ?

- A **pointer** is a **variable** whose **value** is the **address** of another **variable**.
- Uses
 - Pointers allow you to refer to the same space in memory from **multiple locations**. This means that you can update memory in one location and the change can be seen from another location in your program. You will also save space by being able to share components in your data structures.
 - **Dynamic memory allocation**
 - **DMA** allows you to **set array size dynamically** during **run time** rather than at compile time. This helps when the program doesn't know in advance about the **number of items(variable values)** to be stored



1) `malloc()` → Allocates a block of memory of specified size and return the pointer to the beginning of the block. It doesn't initialize the memory.

2) `calloc()` → Allocates memory of array of elements, initializes all bytes to zero and return pointer to the first element.

3) `free()` → Deallocates memory previously allocated by `malloc()`, `calloc()` or `realloc()`.

4) `New`: Allocates memory for a single variable or any array of variables.

5) `Delete`: Deallocates ————— by `new`.


```
#include <iostream>
```

```
using namespace std;
```

```
int main ( )
```

```
{
```

```
    int *ptr = new int;
```

```
    *ptr = 42;
```

```
    cout << "Value : " << *ptr ;
```

```
    delete ptr;
```

```
    int size;
```

```
    cout << "Enter the number of elements:";
```

```
    cin >> size;
```

```
    int* array = new int [size];
```

```
    for(int i=0; i<size; ++i)
```

```
    { array[i] = i*2; }
```

```
    cout << "Array elements:";
```

```
    for (int i=0; i<size; ++i)
```

```
    { cout << array[i] << " "; }
```

```
    cout << endl;
```

```
    delete [] array;
```

```
    return 0; }
```